

SECURITY ISSUES AND DEFENSIVE APPROACHES IN DEEP LEARNING FRAMEWORKS

A project Report submitted in partial fulfilment of the requirements for the

award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

L.Mrudula 221810311014

Saadhvi Cheruku 221810311025

P.Phani Satya Sai 221810311021

K.M.Deekshitha 221810311053

Under the esteemed guidance of

Mr VINAY KUMAR JOSHI

Assistant Professor



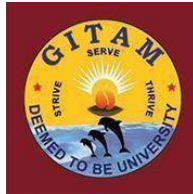
**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING SCHOOL OF TECHNOLOGY**

GITAM

HYDERABAD

APRIL 2022

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GITAM INSTITUTE OF TECHNOLOGY GITAM
(DEEMED TO BE UNIVERSITY)



DECLARATION

We, hereby declare that the project report entitled **“SECURITY ISSUES AND DEFENSIVE APPROACHES IN DEEP LEARNING FRAMEWORKS”** is an original work done in the Department of Computer Science and Engineering, GITAM Institute of Technology, GITAM (Deemed to be University) submitted in partial fulfilment of the requirements for the award of the degree of B.Tech. in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

REGISTRATION NO(S).

221810311014

221810311025

221810311021

221810311053

NAME(S)

L. Mrudula

Saadhvi Cheruku

P.Phani Satya Sai

K.M.Deekshitha

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**GANDHI INSTITUTE OF TECHNOLOGY AND MANAGEMENT
(GITAM)**

(Deemed-to-be-University)



CERTIFICATE

This is to certify that the project report “**SECURITY ISSUES AND DEFENSIVE APPROACHES IN DEEP LEARNING FRAMEWORKS**” is a bonafide record of work carried out by L.Mrudula (221810311014), Saadhvi Cheruku (221810311025), P Phani Satya Sai (221810311021), K.M.Deekshitha (221810311053) in partial fulfilment of the requirements for the award of degree in “Bachelor of Technology” in “Computer Science and Engineering”.

Project Guide

Mr VINAY KUMAR JOSHI
(Assistant Professor)

Head of Department

Dr. PHANI KUMAR
(Professor & HOD)

TABLE OF CONTENTS

1.	Abstract	1
2.	Introduction	2
3.	Literature Review	7
4.	Problem Identification & Objectives	10
5.	System Methodology	11
6.	Overview of Technologies	20
7.	Implementation	26
	7.1 Coding	26
	7.2 Testing	30
8.	Results & Discussions	32
9.	Conclusion & Future Scope	33
10.	References	34

1. ABSTRACT

Deep learning has wide range of applications. But, the security issues in deep learning is stopping it from being widely used. Internal and External attacks on deep learning by attackers impact the society. The report has a description of the deep learning frameworks. The report also includes categorising various attacks on deep learning and counter methods for these attacks. It also discusses about the further development and future scope of the project.

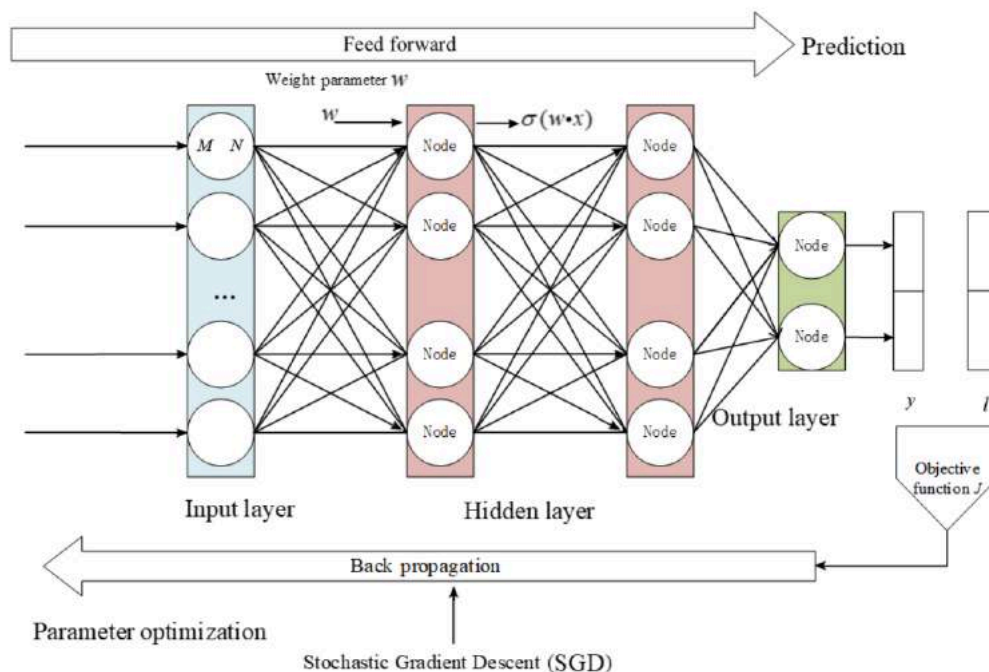
2. INTRODUCTION

2.1 DEEP LEARNING

Deep learning is a subdivision in machine learning. The brain structure inspired the creation of deep learning. Deep learning models can be utilised on plain content, pictures, audio, etc. This allows them to be utilised in numerous domains. In deep learning, the word means the various “layers” in the model

2.1.1 Architecture of the Deep Learning Framework

The Deep Neural Network has two stages. The first stage is the training stage. In this stage the accessible data is used on the network parameters. The second stage is the prediction stage. In this stage, the parameters are used to test on unknown data. In a typical neural network training process parameters are obtained in such a way that the cost work is reduced to minimum. The cost function is used to calculate the error between the estimated measure and the true measure of the data. In order to Complete the training phase of Deep Neural Network, front and back transmissions are required. The input is spread across a layer in order to get the output in the forward stage,. Next, in order to reduce the error between the estimated and the true label, an algorithm called gradient drop is utilised. The results obtained in the second stage are used in the next stage. Here the model handles the output as result.



General DNN training process

2.2 SECURITY ISSUE IN DEEP LEARNING FRAMEWORKS

Numerous Problems from various fields can be solved using the algorithms of deep learning. In addition, many functional programs are operated using deep learning algorithms. While deep learning can bring some comfort, it tends to be more risky. Research conducted in recent years has shown that deep learning is effected by well-designed contradictory samples. These samples can effortlessly deceive a powerful model. Deep learning security issues may occur when an attacker uses a DNN application system that produces conflicting samples or uses the vulnerabilities in the third-party libraries of deep neural network. Using frameworks of deep learning like Caffe, TensorFlow, and Torch, permits the developers to ignore the basic usage details. This increases the efficiency of the applications. The convolution of the framework effects the effectiveness of the model. More the convolution, more the risk. The frameworks mentioned above are integrated into many libraries used by open source companies. The libraries which are using the above mentioned frameworks are at a high probability of being attacked by denial of service, system crashes or escape attacks at network level

The issues in the security of Deep learning will take place if an attacker utilizes a deep neural network application to produce opposing samples or make use of the mediator libraries relying on deep neural network.

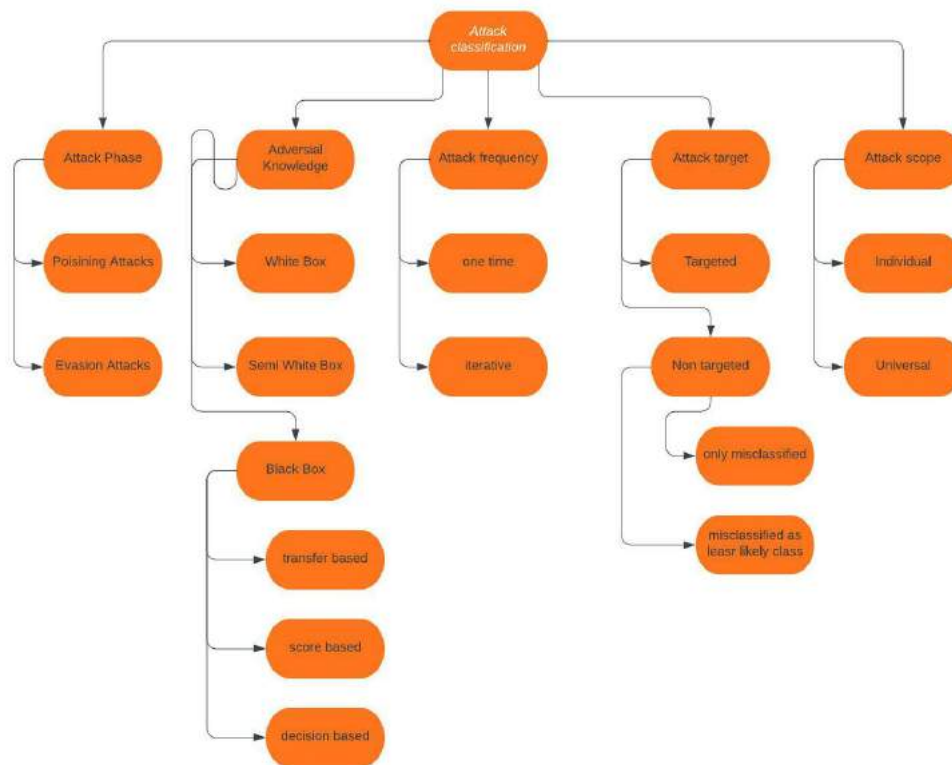
2.2.1 Producing Adversarial Samples

Fast Gradient Signed Method (FGSM) algorithm examines the basis for the existence of conflicting samples, and introduces the method of producing these based on that analysis. The procedure includes a few additional distractions that can't be noticed by the human eye in an image. This has a significant impact on the detector by activating the action.

2.2.2 Vulnerabilities of Deep Learning Frameworks

The procedures for producing contradictory samples are fast gradient signed method and deep fool. Both of these two are white box attacks. To reduce the loss function of a nn, back propagation is used. FGSM attacks go the other way. It adds to the disruption of the soft tissue to produce a resistant sample to enhance the loss of function. This will confuse the neural network model. The framework's complexity determines its effectiveness. If the system is too complex, there is a high risk of accidents.

2.3 TYPES OF ATTACKS ON DEEP LEARNING FRAMEWORKS



Attacks Classification

The deep learning attacks can be classified into various categories and sub categories. Poisoning and Evasion attacks are attacks made in different phases of neural network. The poisoning attack adds conflicting samples along with training sample to effect classification and detect the incorrect label. Evasion attacks utilize contradictory data in the second phase to change the result to an error.

According to information known by the attacker about the system, There can be three types of attacks. First one is white box attacks. In this all the information about the system such as database and algorithm used, the arrangement of each network layer is known to the attacker. Second one is black box attacks. In this the attackers does not have any idea about the system. The last one is semi white box attacks. The attacker only has a part of information about the system in this. A white box is practically not possible to implement.

Further the Black box attacks are categorised into three types. One - Transfer-based. In Transmission-based attack, first the model is trained and then contrasting sample are utilised to strike the selected model. Second - Outcome-based attack. These attacks gains facts within the

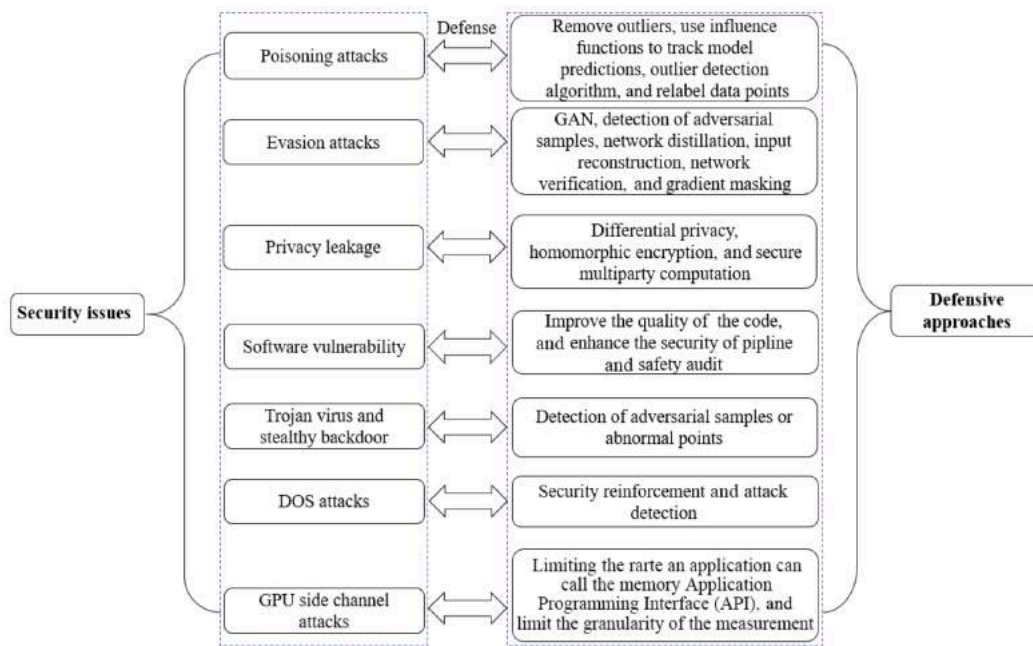
system by manipulating the target system category into input. Third One - Decision-based attacks. These attacks are effective yet very hard to eliminate. In terms of frequency of attacks, attacks are categorised into single and repetitive attacks. Single time attack produce enemy samples only once, while repeated attacks require a more times to upgrade opposing samples. On Comparing both the attacks, it is clear that one-time attacks require less time than repetitive attacks. Although additional disruption is greater in one time attacks. Repeated attacks generate greater outcome. Thus requires more computer means.

Based on which part to target, the attack are categorised into intentional and unintentional. An Attack is called intentional attack if the objective of the attacker is to modify the divider's output into selected tag. In an unintentional attack, the attacker's intention is for the resident to choose any wrong label. In general, unintentional attacks have more triumph rates than intentional attacks. Unintentional attacks can only be distinguished by a negative attack and a very small potential attack.

In terms of the scope of the attack, Individual and universal attacks can be named. Individual attack only needs to fix some features, while a universal attack needs to fix all the features. Thus, the contradictory disturbances caused by individual attacks are less noticeable than those produced by universal.

The model can be destroyed by inserting and fixing the samples in a gradient manner in a white box attack. Since Attackers have a lot of expertise in the domain, the white box attack can be easily spotted. In contrast, a black box attack is hard to implement due to information restrain.

2.4 PROTECTIVE MEASURES IN DEEP LEARNING FRAMEWORKS



Protective Measures

In order to make the most of the deep learning models, many precautions are held opposed to its attacks. Poisonous attacks are used to produce conflicting samples, and the defensive method of such attacks is to remove foreign objects with huge data. Skipping attacks can be put a stop by improving the strength of the class dividers. Encryption algorithms are used to monitor privacy leaks. Malicious software attacks can be prevented using high encrypting quality code and choosing secure mediator libraries.

3. LITERATURE REVIEW

3.1 GEOSPATIAL DATA TO IMAGES: A DEEP-LEARNING FRAMEWORK FOR TRAFFIC FORECASTING

Authors: W. W. Jiang and L. Zhang

Traffic forecasting has been a viable field of research in recent decades, and with the development of deep learning technologies, researchers are trying to use deep learning to achieve greater improvements in traffic forecasting, as has been shown in other research areas, such as speech, image recognition and classification. In this study, we summarize recent activities in which in-depth study methods were applied to geospatial data for traffic forecasting problems. Based on information from previous activities, we also develop a deep learning framework, which converts geospatial data into images, and uses deep learning methods such as Convolutional Neural Network (CNN) and residual networks. . Demonstrating the simplicity and effectiveness of our framework, we present the structure of the New York taxi tax evasion / drop-off problem, and demonstrate that our framework exceeds traditional methods, including Historical Average (HA) and AutoRegressive Integrated Moving Average. (ARIMA).

3.2 IMPROVED DOTA2 LINEUP RECOMMENDATION MODEL BASED ON A BIDIRECTIONAL LSTM

Authors: L. Zhang, C. B. Xu, Y. H. Gao, Y. Han, X. J. Du, and Z. H. Tian,

In recent years, e-sports have developed rapidly, and the industry has produced large amounts of detailed data, and this data is readily available. Because of the above features, data mining and deep learning methods can be used to guide players and develop appropriate game-winning strategies. As one of the world's most popular e-sports events, Dota2 has a large audience and an excellent gaming system. Winning in a game is often associated with a hero's game, and players are often unable to choose the best range to compete. To solve this problem, in this paper, we present an advanced bidirectional Dota2 network model of system recommendations. The model uses the Continuous Bag Of Words (CBOW) model in the Word2vec model to generate hero vectors. The CBOW model can predict the context of a word in a sentence. Thus, the word is transformed into a hero, the sentence into a lineup, and the vector of the word into a hero vector, the model used in this article praises the last hero according to the first four heroes selected, thus solving a series of recommendations. problems.

3.3 ADVERSARIAL EXAMPLES: ATTACKS AND DEFENSES FOR DEEP LEARNING

Authors: X. Y. Yuan, P. He, Q. L. Zhu, and X. L. Li

With rapid development and significant success in a wide range of applications, deep learning is used in many of the most important areas of security. However, deep neural networks have recently been identified as vulnerable to well-designed input samples, called contrasting models. Contradictory examples are invisible to the human eye but can easily deceive deep neural networks in the test / extraction phase. The vulnerability of conflicting models becomes one of the major risks of using deep neural networks in critical security areas. Therefore, attacks and defenses against conflicting examples attract a lot of attention. In this paper, we review the latest findings from conflicting models of deep neural networks, summarize the methods for generating conflicting models, and raise the taxes for these methods. Under the taxonomy, applications for conflicting models are investigated. We also describe steps to resist conflicting examples and explore potential challenges and solutions.

3.4 A MEMORY-RELATED VULNERABILITY DETECTION APPROACH BASED ON VULNERABILITY FEATURES

Authors: J. C. Hu, J. F. Chen, L. Zhang, Y. S. Liu, Q. H. Bao, H. Ackah- Arthur, and C. Zhang

Improving secure software systems is a major challenge in the software industry due to bugs or weaknesses that pose a risk to the software system. To address this challenge, researchers often use risk code features to improve risk identification. In addition to the success of these strategies, the available studies have focused on conceptual definitions without the precise definition of risk factors. In this study, we present the novel and How to Recognize Effective Memory-Based Dangers using Risk Factors (MRVDAVF). Our framework uses three different strategies to improve risk identification. In the first section, we introduce the advanced Control Flow Graph (CFG) and Pointer-related Control Flow Graph (PCFG) to define the specific risk factors, including memory leakage, duplication, and post-release use. Subsequently, two algorithms, namely the Vulnerability Judging algorithm based on the Vulnerability Feature (VJVF) and the Feature Judging (FJ) algorithm, are used to detect memory-related risks. Finally, the proposed model is validated using three test cases found in the Juliet Test Suite. The test results show that the proposed method is feasible and effective.

3.5 NEURAL NETWORKS INTRIGUING PROPERTIES

Authors: C. Szegedy, W. Zaremba, I. Sutskever I, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus,

Deep neural networks are the most expressive models that have recently acquired artistic performance in speech and visual acuity activities. While their explanation is the reason for their success, it also causes them to learn inexplicable solutions that can have conflicting areas. In this paper we report two such aspects. First, we find that there is no difference between each high-level units and random line combinations of high-level units, depending on the various methods of unit analysis. It suggests that space, instead of individual units, contains semantic information in higher layers of sensory networks. Second, we find that deep neural networks that read output input maps do not keep up with critical expansion. Specifically, we find that we can cause a network to misinterpret an image by using some invisible detection, which is detected by increasing the network's error guess. Additionally, the specific nature of this interference is not a random artifact of learning: the same interference can cause a different network, trained in a different subset of the database, to incorrectly separate the same input.

4. PROBLEM IDENTIFICATION & OBJECTIVES

4.1 PROBLEM DEFINITION

While deep learning can bring some comfort, it tends to be more risky. Research conducted in recent years has revealed that deep learning is at the risk of clever contradictory samples. These can effortlessly deceive a good designed model.

4.2 OBJECTIVE OF PROJECT

We propose a comprehensive approach to classify various security problems faced by deep learning and provide corresponding precaution measures to protect the deep learning programs.

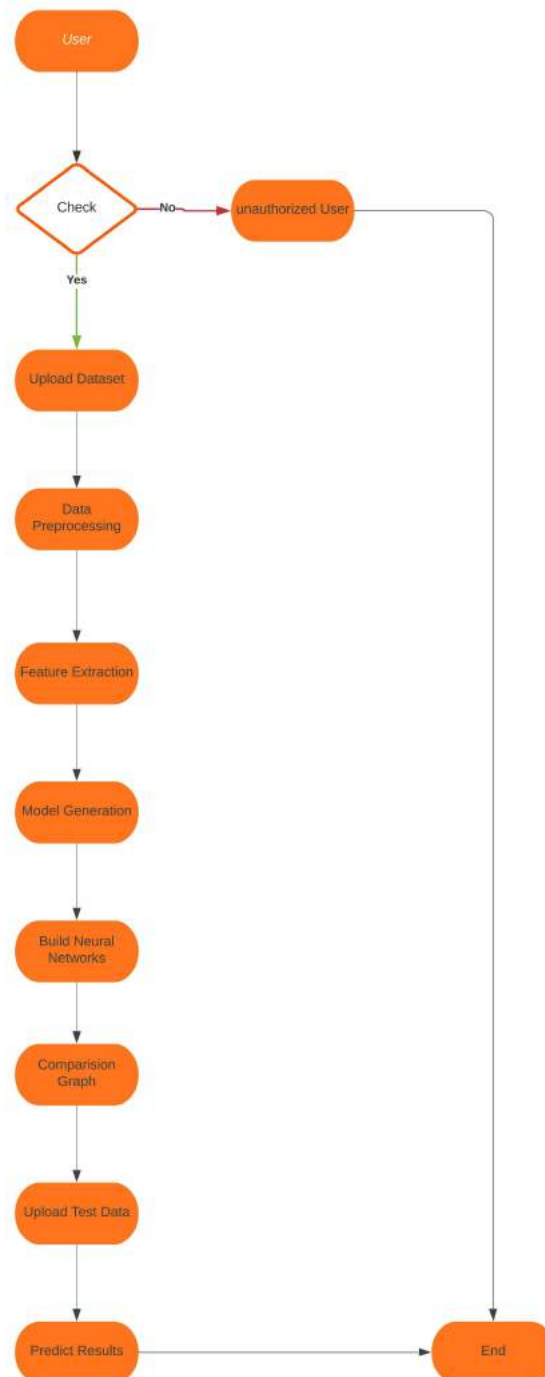
4.3 TECHNOLOGIES USED

1. Python 3.7
2. Scikit – learn
3. Numpy
4. Tensorflow
5. Matplotlib
6. Pandas

5. SYSTEM METHODOLOGY

5.1 FLOWCHART

A flowchart is a diagram showing a process of a project



5.2 UML DIAGRAMS

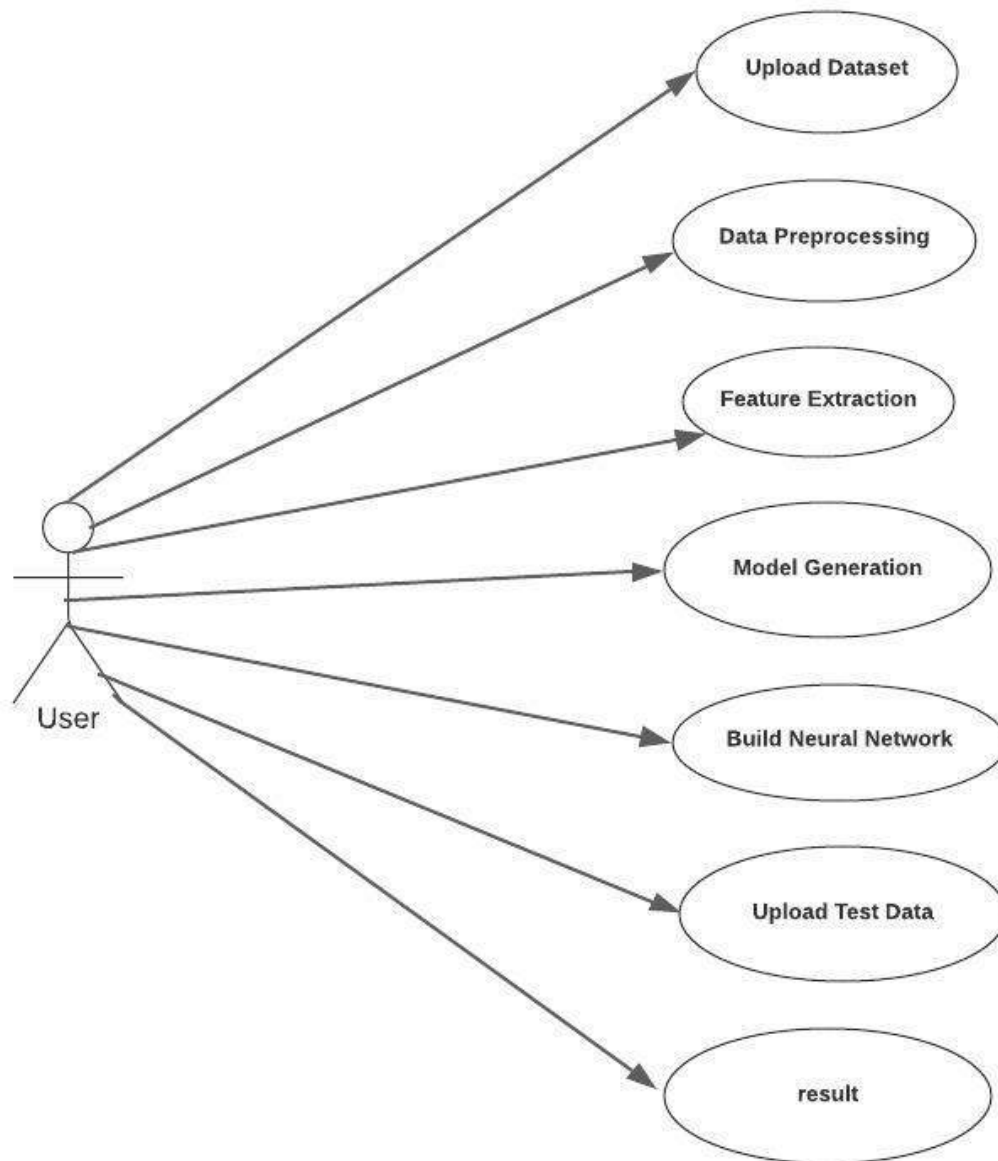
- UML represents Unified Modeling Language.
- UML is used for interpreting, envisioning, constructing, & writing artifacts for software applications.
- Object Management Group created UML in January 1997.
- Software Blueprints can be designed using UML.
- Though it is not a programming language, it is used as a medium to design codes in different programming languages.

UML Diagrams can be categorised into:

1. Use case Diagram
2. Deployment Diagram
3. Class Diagram
4. State Diagram
5. Sequence Diagram
6. Collaboration Diagram
7. Activity Diagram
8. Component Diagram
9. Object Diagram

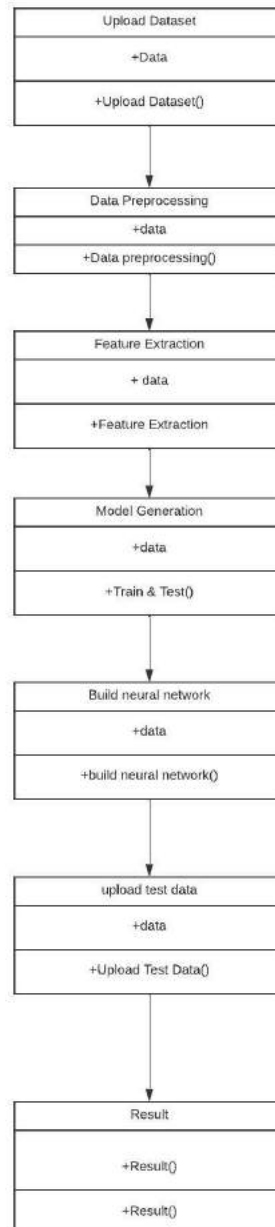
5.2.1 Use Case Diagram

A symbolic representation of all possible relations the user can have with the given product.



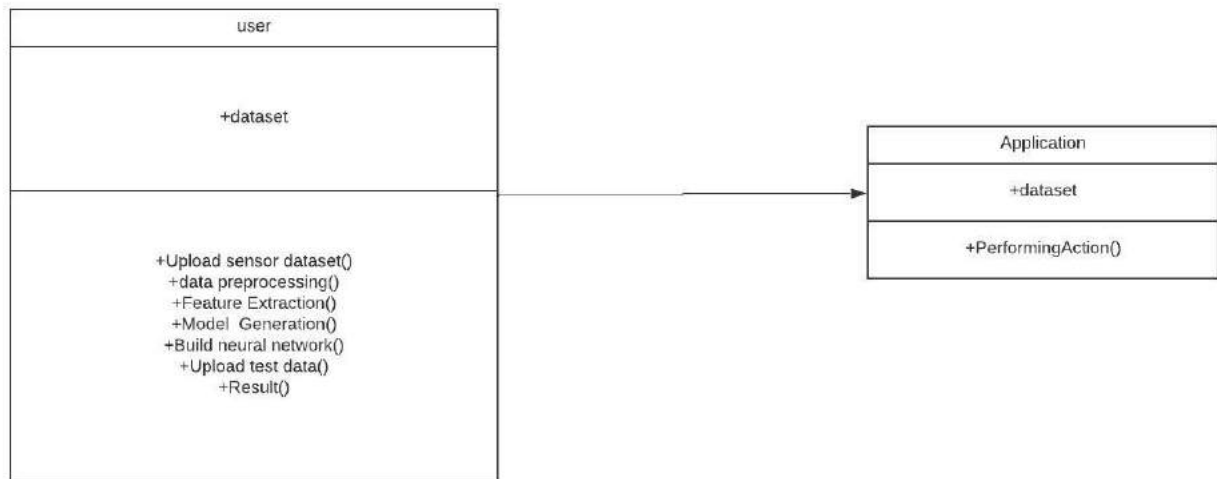
5.2.2 Class Diagram

- A symbolic portrayal of the static view of the product.
- All the attributes and operations of a class are represented using class diagram .
- It can directly be mapped with Object Oriented Languages.



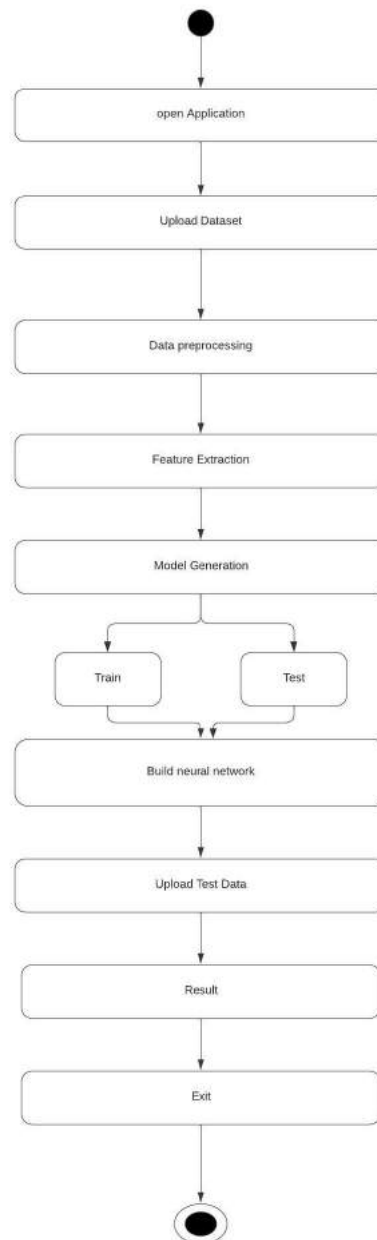
5.2.3 Object Diagram

- An instance of a class is represented using Object Diagram.
- These are basically obtained from class diagrams.



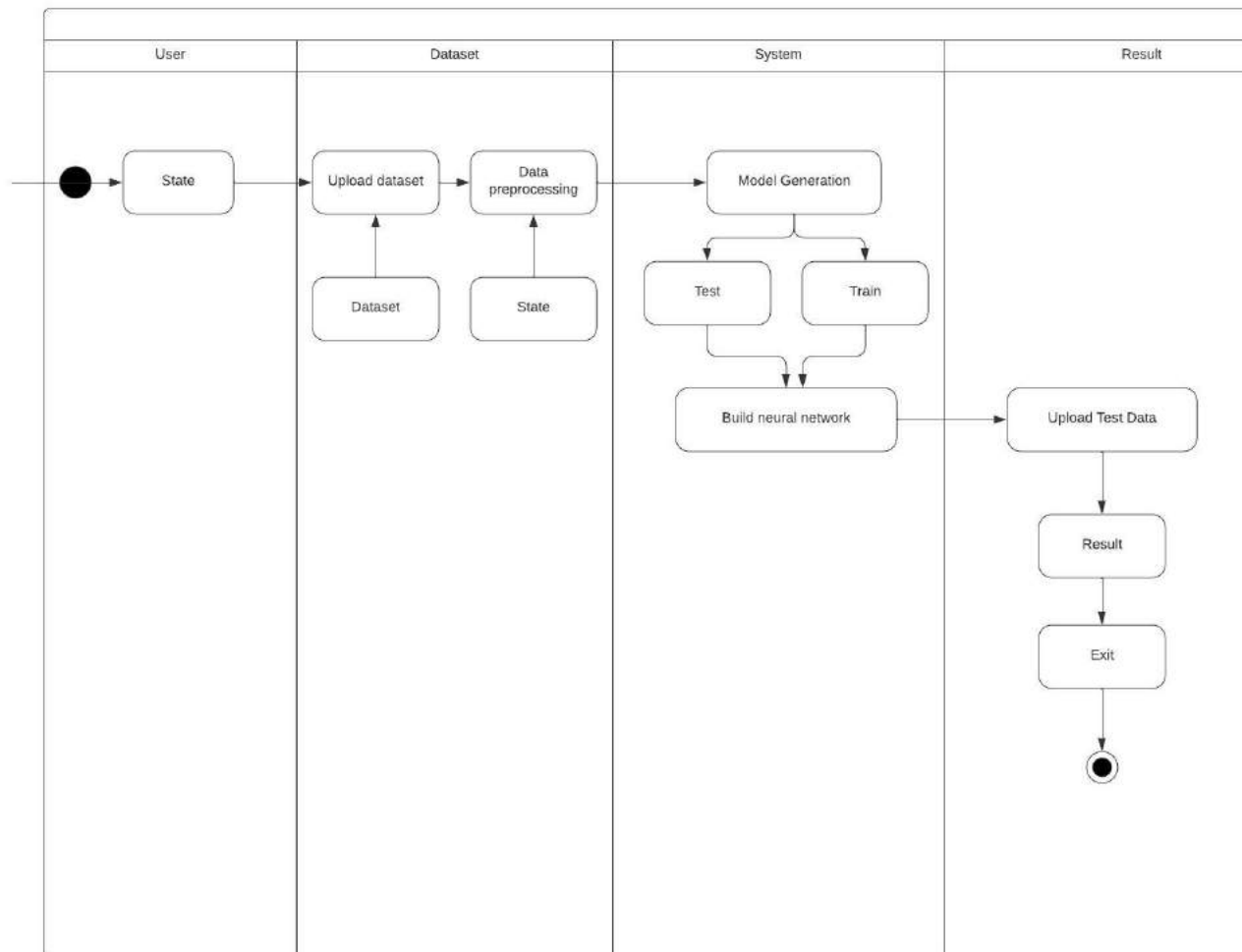
5.2.4 State Diagram

- State Diagrams can be defined as a description of a class's behaviour
- It is also called as State chart diagram
- It reflects various changes in the class environment.



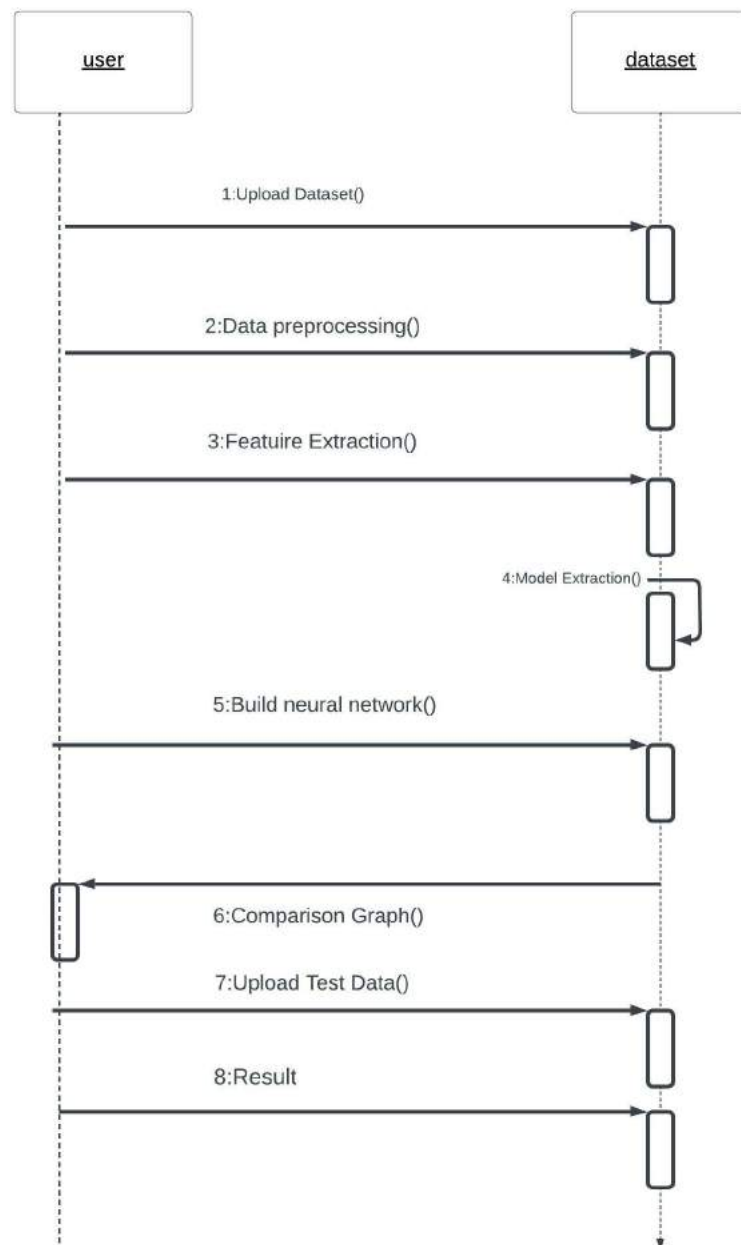
5.2.5 Activity Diagram

- Activity diagram is also a behaviour diagram
- It represents a product's behaviour
- It shows the control flow from the starting point to the end point showing the various decision-making mechanisms present during the operation.



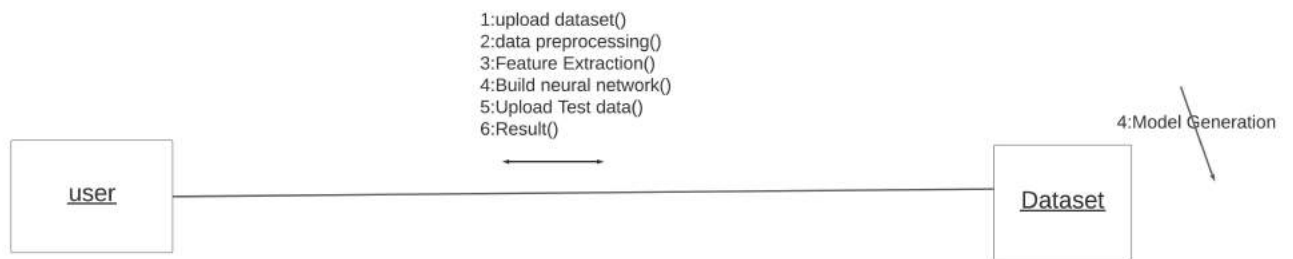
5.2.6 Sequence Diagram

- Sequence Diagram is an interactive diagram
- explains how the collections works — and in what order



5.2.7 Collaboration Diagram

- Collaboration Diagram shows the relationship between objects in a system.
- It shows the structure of object in the system instead of flow of messages.



6. OVERVIEW OF TECHNOLOGIES

6.1 PYTHON

It is a famous programming language. Guido van Rossum was its creator. It got published in the year 1991. It is utilised in many different domains such as server-side development, machine learning, maths, software development, scripting, etc.



6.1.1 Advantages of Python:

Python is a versatile language with many pros. Other languages can extend python language according to their requirement. Example: Some of the code can be written in other languages like c or Java etc. This feature is quite handy for developer . It is a simple language with syntax almost similar to English language. It is easy to grasp, figure out, and code. Hence It is a beginner friendly language. Curly braces are not needed to define blocks. This feature increases the readability of the code. Python is freely available and open source. As compared to compiled languages, Debugging is much easy.

6.1.2 Disadvantages of Python:

Python is slowly executed because it is interpreted. Python is not used on the client-side. Python can raise run time errors as it is a dynamically-typed language

6.2 TENSORFLOW

TensorFlow is a free of charge and is an open source software that is generally used for machine learning and artificial intelligence. This software has been developed by the Google Brain team with an intention to use it within the company. Apache 2.0 released the software on 9 November 2015 under open source license. It has a comprehensive, adaptable tools, libraries and resources. This helps the researchers further enhance the machine learning and developers to develop powerful machine learning applications.



It builds and trains machine learning models using a prominent and quick application programming interface called Keras with an ambitious function. This produces a faster model duplication & eliminates error easily. The model can be easily trained and used in the cloud, device or browser irrespective of the language being used. It takes new ideas from concept and transforms them to high-end models, and to publish quickly.

6.3 NUMPY

NumPy is one of the libraries of Python programming language. It attaches aid for the large, multilateral members and matrices.

It Contains a variety of features including: Powerful N-dimensional List, Complex (broadcast) functions, C / C ++ integration tools with Fortran code etc. Numpy is utilised as a multilateral Vessel for standard information. NumPy permits us to use wrong data types. This to merge seamlessly and quickly with a variety of information.

NumPy has numerous functions to perform mathematical calculations. Examples are random number generators and more. NumPy is compatible with distinct hardware and software. It can do its best with distributed space, graphical processing units, and many libraries. NumPy's syntax can be accessed by programmers of any field or expertise level. GitHub was used to develop and maintain Numpy. NumPy core is coded in C language.



6.4 PANDAS



Pandas is a software mainly used in the area of data for its manipulation, storage, analysis etc. It is one of the libraries in the Python programming language written in python itself. It contains numerous data structures and functions for managing tables and series. It is free and an open source software published under three-clause BSD license. Domains Like finance, statistics, softwares etc can make use of pandas to improve productivity.

It Provides flexibility to the data sets. Columns can be added and removed from data structures by size. Combining or converting data with a dynamic group with an engine that allows segmentation-integration tasks into data sets. High performance integration and integration of data sets. Highly advanced for performance, with key coding methods written Python or C.

6.5 MATPLOTLIB

Matplotlib is a library in python programming language programming. It is an extension of NumPy. It also contains an application programming interface focused on object implanting in projects using common purpose graphical user interface tools such as Qt, Tkinter, GTK, wxPython. With a few lines of codes, graphical representations like bar charts, graphs, histograms etc can be generated. Matplotlib is an amazing Python library of 2D frames. Matplotlib is a database for displaying multiple platform data.



Matplotlib makes simple things easier and harder. Can be used to create publishing quality articles, Create interactive calculations that bring zoom, generosity etc.

6.6 SCIKIT – LEARN



Scikit-learn is a free software written in python programming language. Its applications are mainly found in machine learning domain. It includes algorithms for segmentation, deceleration and integration. Scikit-learn is extremely convenient and robust library in Python. Efficient machine learning tools can be selected from this library which help in splitting, merging etc

7. IMPLEMENTATION

7.1 CODING

7.1.1 Data Loading

- The dataset is uploaded using the method `read_csv()`

```
raw_data = pd.read_csv("Malware dataset.csv")
raw_data.head()
```

	hash	millisecond	classification	state	usage_counter	prio	static_prio	normal_prio	policy	vm_pgoff	...
0	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	0	malware	0	0	3069378560	14274	0	0	0	...
1	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	1	malware	0	0	3069378560	14274	0	0	0	...
2	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	2	malware	0	0	3069378560	14274	0	0	0	...
3	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	3	malware	0	0	3069378560	14274	0	0	0	...
4	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	4	malware	0	0	3069378560	14274	0	0	0	...

5 rows × 35 columns

7.1.2 Data Preprocessing

- The Missing values in the collected dataset can make the model inconsistent.
- To eliminate the inconsistency and improve the effectiveness of the model, Preprocessing of data is done
- we use a function called `map` for this purpose

```
#Start Processing
data = raw_data
```

```
data["classification"].value_counts()
```

```
benign      50000
malware     50000
Name: classification, dtype: int64
```

```
data['classification'] = data.classification.map({'benign':0, 'malware':1})
data.head()
```

	hash	millisecond	classification	state	usage_counter	prio	static_prio	normal_prio	policy	vm_pgoff	...
0	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	0	1	0	0	3069378560	14274	0	0	0	...
1	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	1	1	0	0	3069378560	14274	0	0	0	...
2	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	2	1	0	0	3069378560	14274	0	0	0	...
3	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	3	1	0	0	3069378560	14274	0	0	0	...
4	42fb5e2ec009a05ff5143227297074f1e9c6c3ebb9c914...	4	1	0	0	3069378560	14274	0	0	0	...

5 rows × 35 columns

7.1.3 Model Selection

- Machine learning recognises patterns and predicts suitable results after understanding those patterns.
- An ML model performance enhances from each trail
- Splitting the data into training and test sets helps in evaluating the efficiency of the model.
- we split 70% of dataset into the Training set & the remaining 30% into the Test set.

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(X,Y,test_size=0.2,random_state=1)
```

7.1.4 Data Normalization

- Data collected can have features in various ranges
- To prepare the data to be used it is important to bring all of it to a common scale
- This can be done by using normalization

```
# Data normalization
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```


7.1.5 Creating a Sequential Model

- Sequential model is created using Sequential() method.
- Then Layers are added to the sequential model
- Output data of one layer is given as input to the next layer until the final layer is reached

```
import tensorflow as tf

#Number of attributes
input_size = 27

#Number of Outputs
output_size = 2

# Use same hidden layer size for both hidden layers. Not a necessity.
hidden_layer_size = 50

# define how the model will look like
model = tf.keras.Sequential([
    tf.keras.layers.Dense(hidden_layer_size, input_shape=(input_size,)), activation='relu'), # 1st hidden layer
    tf.keras.layers.Dense(hidden_layer_size, activation='relu'),
    tf.keras.layers.Dense(hidden_layer_size, activation='relu'),
    tf.keras.layers.Dense(hidden_layer_size, activation='relu'),
    tf.keras.layers.Dense(hidden_layer_size, activation='relu'),
    tf.keras.layers.Dense(output_size, activation='softmax') # output layer
])

model.summary()
```

7.1.6 Compiling the Model

- Keras has a function called compile() to compile the model.
- Compile method has different arguments.
- The important arguments in compile() method are: loss function, Optimizer, metrics
- Loss function: finds deviation in the learning process
- Optimizer: the input weights are made effective by comparing the estimated and the loss function.
- Metrics: estimates the efficiency of a model

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
#from keras.optimizers import SGD
#opt = SGD(lr=0.01)
#sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
#model.compile(optimizer = sgd, loss = "sparse_categorical_crossentropy", metrics=['accuracy'])
```

7.1.7 Model Training

- fit() method is used for training the models.
- a measure on how well a machine learning model generalises similar data to that on which it was trained.

```
result = model.fit(x=x_train,
                   y=y_train,
                   batch_size=batch_size,
                   epochs=max_epochs,
                   verbose=1,
                   #callbacks=[early_stopping],
                   validation_split=0.2)
```

7.1.8 Model Training Using SGD

- Instead of using Adam optimizer, this time the model is compiled using SGD optimizer.
- Although Adam converges faster, SGD generalises better than Adam and thus results in improved final performance.
- Accuracy is increased.

```
from keras.optimizers import SGD
sgd = SGD(lr=0.001, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(optimizer = sgd, loss = "sparse_categorical_crossentropy", metrics=['accuracy'])
```

7.2 TESTING

Testing is the process of finding any fault in a product. It lay outs a method to evaluate the performance of a finished product. There are different types of tests.

7.2.1 Unit Testing

Unit testing included testing each unit or part of a software program. It is the first step in testing performance. The purpose of the unit test is to verify the components of the unit for its operation.

7.2.2 Integration Testing

The test to integrate the second level of the software testing process comes after the unit test. In this test, individual units or software components are tested in a group. The focus of the test level integration is to identify errors during interactions between integrated components or units.

7.2.3 Functional Test

It is a testing method used to confirm software performance, even if the function works in accordance with a set requirement. In in tests, Evaluation is done by assigning a value to the output, and verifying the actual output with the expected value. Black box test is introduced to ensure that the performance of a system is working in the way we expect it to. Done to verify app performance. The inspector should check the system only rather than the system.

7.2.4 System Test

System Testing is performed on a completed product to test the compliance needs. In the system test, the approved components of the integration test are considered to be included.

7.2.5 White Box Testing

Examines coding and software that are based on preset input and the expected output. For this kind of testing planning is required to design test cases. The main goal of the white box test is to concentrate on the input and output of the software and to strengthen the software's reliability.

7.2.6 Black Box Testing

In the black box check, the performance of software is verified without checking its structure or coding. The information provided by the customer is the main source of black box testing. In this method, the tester chooses a function and assigns an input value to test the performance, and sees if the result is as expected.

Test Results:

All the test cases mentioned above passed successfully.

8. RESULTS AND DISCUSSIONS

8.1 Test Accuracy With Adam Optimizer

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print('\nTest loss: {0:.6f}. Test accuracy: {1:.6f}%'.format(test_loss, test_accuracy*100.))

625/625 [=====] - 1s 2ms/step - loss: 9.6661e-04 - accuracy: 0.9998

Test loss: 0.000967. Test accuracy: 99.980003%
```

8.2 Test Accuracy With SGD Optimizer

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print('\nTest loss: {0:.6f}. Test accuracy: {1:.6f}%'.format(test_loss, test_accuracy*100.))

625/625 [=====] - 1s 2ms/step - loss: 6.7222e-04 - accuracy: 0.9998

Test loss: 0.000672. Test accuracy: 99.984998%
```

9. CONCLUSION & FUTURE SCOPE

9.1 CONCLUSION

The project first describes about the structure and principles behind the deep learning model and then discusses about the attacks on deep learning frameworks along with preventive measures. The training and prediction stages in the deep learning model can be better understood by studying combative algorithms. This report discusses about various attacks that effected the security of deep learning in the recent time. Preventive measures for the above mentioned attacks are also stated. The security of deep learning can be compromised by slight deviations. This shows that the deep learning can be effected by many factors. It's one of the reasons that is impeding the deep learning from going to a great extent. Few issues such as object recognition can be solved using eep learning with a good accuracy. But the actual problem rises if the model has to deal with interacting with external world. This scenario can lead to a lot of errors and mistakes. Artificial Intelligence Bottleneck is the term given to this particular situation. So from our analyses, we conclude that though deep learning is a powerful domain & can be used in a number of domains, its practical applications in the real world can only happen if more focus is kept on making it more secure.

9.2 FUTURE SCOPE

Deep learning is an upcoming field. Researchers are constantly looking for any vulnerabilities and respective preventive measures of deep learning so that it can be applied in the real world. It is a constantly developing field and has a long term development scope.

The extensive presence of conflicting information helps to improve the efficiency of algorithms of deep learning. The results of deep learning predictors have significant deviations from minor disruptions, indicating that deep learning algorithms require longer. Progress continues, and current progress remains incomplete and immature.

10. REFERENCES

- [1] W. W. Jiang and L. Zhang, Geospatial data to images: A deep-learning framework for traffic forecasting, *Tsinghua Science and Technology*, vol. 24, no. 1, pp. 52–64, 2019.
- [2] L. Zhang, C. B. Xu, Y. H. Gao, Y. Han, X. J. Du, and Z. H. Tian, Improved Dota2 lineup recommendation model based on a bidirectional LSTM, *Tsinghua Science and Technology*, vol. 25, no. 6, pp. 712–720, 2020.
- [3] H. M. Huang, J. H. Lin, L. Y. Wu, B. Fang, Z. K. Wen, and F. C. Sun, Machine learning-based multi-modal information perception for soft robotic hands, *Tsinghua Science and Technology*, vol. 25, no. 2, pp. 255–269, 2020.
- [4] X. Y. Yuan, P. He, Q. L. Zhu, and X. L. Li, Adversarial examples: Attacks and defenses for deep learning, *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 9, pp. 2805– 2824, 2019.
- [5] J. C. Hu, J. F. Chen, L. Zhang, Y. S. Liu, Q. H. Bao, H. Ackah-Arthur, and C. Zhang, A memory-related vulnerability detection approach based on vulnerability features, *Tsinghua Science and Technology*, vol. 25, no. 5, pp. 604–613, 2020.
- [6] C. Szegedy, W. Zaremba, I. Sutskever I, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, Intriguing properties of neural networks, *arXiv preprint arXiv: 1312.6199*, 2013.
- [7] A. Athalye, N. Carlini, and D. Wagner, Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples, *arXiv preprint arXiv: 1802.00420*, 2018.
- [8] Y. T. Xiao, C. M. Pun, and J. Z. Zhou, Generating adversarial perturbation with root mean square gradient, *arXiv preprint arXiv: 1901.03706*, 2019.
- [9] I. J. Goodfellow, J. Shlens, and C. Szegedy, Explaining and harnessing adversarial examples, *arXiv preprint arXiv: 1412.6572*, 2014.
- [10] J. W. Su, D. V. Vargas, and K. Sakurai, One pixel attack for fooling deep neural networks, *IEEE Trans. Evol. Comput.*, vol. 23, no. 5, pp. 828–841, 2019.
- [11] W. He, J. Wei, X. Y. Chen, N. Carlini, and D. Song, Adversarial example defense: Ensembles of weak defenses are not strong, in *Proc 11th USENIX Workshop on Offensive Technologies*, Vancouver, Canada, 2017.

- [12] G. W. Xu, H. W. Li, H. Ren, K. Yang, and R. H. Deng, Data security issues in deep learning: Attacks, countermeasures, and opportunities, *IEEE Comm. Mag.*, vol. 57, no. 11, pp. 116–122, 2019.
- [13] M. I. Tariq, N. A. Memon, S. Ahmed, S. Tayyaba, M. T. Mushtaq, N. A. Mian, M. Imran, and M. W. Ashraf, A review of deep learning security and privacy defensive techniques, *Mobile Inf. Syst.*, vol. 2020, p. 6535834, 2020.
- [14] H. Bae, J. Jang, D. Jung, H. Jang, H. Ha, and S. Yoon, Security and privacy issues in deep learning, *arXiv preprint arXiv: 1807.11655*, 2018.
- [15] S. L. Qiu, Q. H. Liu, S. J. Zhou, and C. J. Wu, Review of artificial intelligence adversarial attack and defense technologies. *Appl. Sci.*, vol. 9, no. 5, p. 909.